

Chunking Techniques

Wednesday, July 29, 2026 7:55 PM

Technique	How it Works	Pros	Cons	When to Use
Fixed-size chunking	Split every N tokens (e.g., 500)	Very fast	Breaks context	Small/simple documents
Sliding window	500 tokens with 100-token overlap	Preserves context	Duplicate data	Most production RAG systems
Sentence-based	Split at sentence boundaries	Better readability	Uneven chunk sizes	Articles, reports
Paragraph-based	One paragraph per chunk	Preserves meaning	Some chunks may be too long	PDFs with clear formatting
Semantic chunking	Split when topic changes	High retrieval quality	Computationally expensive	High-quality enterprise search
Hierarchical chunking	Document → Section → Paragraph	Excellent context	More complex	Large manuals, books

Fixed-size Chunking:

- **Straight forward** approach to chunking.
- Simply decide the no.of tokens in our chunk and use this number to break up our documents into fixed size chunks.
- **Max context window size** of the embedding model (such as **1024** for **llama-text-embed-v2**, **8196** for **text-embedding-3-small**).

Notes: Fixed-sized chunking will be the best path in most cases, and we recommend starting here and iterating only after determining it insufficient.

Simple Sentence and Paragraph splitting

- **Naive splitting:** This approach would be to split sentences by periods ("."), new lines, or white space.
- **NLTK:** It provides a trained sentence tokenizer that can split the text into sentences, helping to create more meaningful chunks.
- **spaCy:** It offers a sophisticated sentence segmentation feature that can efficiently divide the text into separate sentence

Recursive Character Level Chunking:

LangChain implements a [RecursiveCharacterTextSplitter](#) that tries to split text using separators in a given order. The default behavior of the splitter uses the ["\n\n", "\n", " ", ""] separators to break paragraphs, sentences and words depending on a given chunk size.

Document structure-based chunking

When chunking large documents such as PDFs, DOCX, HTML, code snippets, Markdown files and LaTeX,

specialized chunking methods can help preserve the original structure of the content during chunk creation.

- **PDF** documents contain loads of headers, text, tables, and other bits and pieces that require preprocessing to chunk.
- **HTML**, from scraped web pages can contain tags (<p> for paragraphs, or <title> for titles) that can inform text to be broken up or identified, like on product pages or blog posts. Roll your own parser.
- **Markdown** is a lightweight markup language commonly used for formatting text. By recognizing the Markdown syntax (e.g., headings, lists, and code blocks), you can intelligently divide the content based on its structure and hierarchy, resulting in more semantically coherent chunks.
- **LaTeX** is a document preparation system and markup language often used for academic papers and technical documents. By parsing the LaTeX commands and environments, you can create chunks that respect the logical organization of the content (e.g., sections, subsections, and equations), leading to more accurate and contextually relevant results.

Semantic Chunking

A new experimental technique for approaching chunking was first introduced by [Greg Kamradt](#)

if you're building an application with LLMs, you most likely already have the ability to create embeddings - and embeddings can be used to extract the semantic meaning present in your data. This semantic analysis can be used to create chunks that are made up of sentences that talk about the same theme or topic.

Contextual Chunking with LLMs

Sometimes, it's not possible to chunk information from a larger complex document without losing the context entirely. This can happen when the documents are many hundreds of pages, change topics frequently, or require understanding from many related portions of the document. Anthropic introduced [contextual retrieval in 2024](#) to help address this problem.